

HealthClaimHardened Z7 Validation

Read-only validation report | 4 October 2026

FINAL DECISION: PASS WITH OBSERVATIONS

Scope and environment

No source code, tests, architecture, Core, Gateway, Fabric, or IPFS files were changed. This report is the only artifact created for this validation. HealthClaimCore-v1 remains frozen.

Item	Observed environment
OS	Ubuntu 24.04.5 LTS; Linux 6.17.0-1032-oem; x86_64
CPU	AMD Ryzen 7 PRO 8840HS with Radeon 780M Graphics; 8 cores / 16 threads
RAM	27 GiB total reported by free -h
Go	go1.25.8 linux/amd64; module go 1.25.7, toolchain go1.25.8
gnark	v0.16.3
gnark-crypto	v0.21.0

Commands and execution

Ran in zkp/healthclaimhardened: go version; go mod verify; GOCACHE=/tmp/healthclaim-hardened-cache go test -count=1 -v ./...; and GOCACHE=/tmp/healthclaim-hardened-cache go test -count=1 -run '^\$' -bench 'BenchmarkProof(Generation|Verification)\$' -benchtime=10x -benchmem. Go reported version 1.25.8; all modules verified.

Build and tests passed. The tests compile the R1CS, solve a valid witness, run 24 adversarial unsatisfied-witness cases, generate and verify a Groth16 proof, reject mutations to all 13 public inputs, validate the trust-anchor boundary, check canonical context values, and compare the deterministic vector. Six top-level tests and 37 nested cases passed; no failures.

Exact circuit counts

22,021 R1CS constraints; 13 public variables; 31 private variables. The public-variable count is confirmed by a compiled-circuit assertion and test log. Serialized proof size measured by gnark Proof.WriteTo: 164 bytes.

Measured benchmark

Metric	Z6 baseline	Z7 actual	Difference
Constraints	19,657	22,021	+2,364 (+12.0%)
Proof generation	64.80 ms/op	70.330 ms/op	+5.530 ms (+8.5%)
Proof verification	0.635 ms/op	0.658 ms/op	+0.023 ms (+3.6%)
Proof serialized size	Not reported	164 bytes	Z6 size unavailable
Generation allocs / bytes	Z6: 2,644 / 21,163,069	Z7: 2,623 / 21,693,652	10x benchmark run
Verification allocs / bytes	Z6: 165 / 30,384	Z7: 180 / 31,876	10x benchmark run

Timing is a local 10-iteration microbenchmark on the same host and method as Z6. The Z7 measured figures in this report are from the requested validation run, not earlier implementation-run numbers.

Independent relation review

The circuit retains the Core relation and adds a Poseidon2 context commitment over the ordered tuple: tag 106, protocol domain, challenge, recipient ID, deployment ID, policy version, root version, policy commitment, registry root, authority key coordinates, nullifier, and claim amount. The circuit constrains ProtocolDomain to 202610041. Root and policy versions have 64-bit range constraints. ContextDigest rejects noncanonical BN254 inputs and oversized versions.

Property	Code-level validation
Challenge binding	Challenge is a public input and part of ContextCommitment. A changed challenge makes the old witness unsatisfied; proof verification also rejects the changed public input.
Recipient binding	RecipientID is included in the context tuple; mutation tests fail in-circuit and at Groth16 verification.
Deployment/domain binding	DeploymentID enters context. ProtocolDomain is both context-bound and asserted equal to 202610041. Mutations are rejected.
Root-version binding	RootVersion is 64-bit constrained and context-bound with RegistryRoot. Both public-input mutation and changed witness tests reject.
Policy-version binding	PolicyVersion is 64-bit constrained and context-bound with PolicyCommitment. Mutations reject.
Context commitment	Circuit recomputes the fixed ordered Poseidon2 tuple and asserts equality to public ContextCommitment.
Nullifier	Poseidon2(tag 105, PatientSecret, path-derived little-endian RecordIndex); asserted equal to public Nullifier and included in context.
Merkle membership	Eight Boolean direction selectors and siblings reconstruct the public RegistryRoot. Root and sibling mutations fail.
Authority signature	EdDSA verifies over the committed leaf using MiMC challenge hash; authority key and signature R are constrained on-curve.
Policy commitment	Recomputed over PolicyID, lab min/max, policy max claim, and the ordered four-diagnosis set; asserted equal to the public commitment.

External trust anchor and replay

The self-selected-authority-key test intentionally constructs a valid signature under an attacker key and recomputes context; the circuit is satisfiable. Thus the circuit proves a signature under the supplied key, not that the key is a registered authority. The verifier must anchor authority key/status, accepted root/version, policy commitment/version, deployment, recipient, and the challenge against trusted state. Issuer registration must also reject identity and enforce subgroup policy.

Replay is not claimed as circuit-only protection. Challenge freshness, acceptance window, and one-time use are verifier responsibilities. UsedNullifiers storage is external. The nullifier remains stable across requests only if the registry preserves a record's index; the circuit does not prevent the same record from being reinserted or reindexed under another root.

Adversarial circuit tests

For the following witness mutations, expected result was unsatisfied witness; actual test result was rejection in every case. The precise Go subtest names are in parentheses.

Mutation tested	Security property demonstrated
Challenge (wrong challenge)	Request challenge binding
Recipient (wrong recipient)	Recipient binding
Domain (wrong domain)	Fixed protocol domain and separation
Deployment (wrong deployment context)	Deployment/channel binding
Root version (wrong root version)	Versioned root context
Policy version (wrong policy version)	Versioned policy context
Public root (wrong Merkle root)	Root statement and path binding
Merkle sibling (wrong registry root path)	Authenticated path
Policy digest (wrong policy commitment)	Policy recomputation and context
Nullifier (wrong nullifier)	Record-derived nullifier equality
Authority X (wrong authority public key)	Supplied signing key is statement-bound
Signature S (invalid authority signature)	Signature validity over leaf
Direction selector = 2 (non-Boolean Merkle direction)	Boolean Merkle choice
Diagnosis 41 -> 42 (diagnosis not covered)	Covered-set membership
Lab 6 -> 2 / 9 (below / above range)	Both lab bounds
Claim 900 -> 1001 (claim above ceiling)	Claim <= coverage ceiling
Ceiling 1000 -> 1201 (coverage ceiling exceeds policy maximum)	Ceiling <= policy maximum
Lab / claim / ceiling = 2^64	64-bit value bounds
Root / policy version = 2^64	64-bit version bounds
Context digest changed (wrong context digest)	Context recomputation equality

Groth16 public-input mutation tests

All 13 changed-public-input subtests rejected verification of the original proof: RegistryRoot, RootVersion, PolicyCommitment, PolicyVersion, ClaimAmount, AuthorityKeyX, AuthorityKeyY, Nullifier, ProtocolDomain, Challenge, RecipientID, DeploymentID, and ContextCommitment.

Other tests and vector reproducibility

The native ContextDigest helper rejects a negative scalar, a challenge equal to the BN254 scalar modulus, and a policy version above 64 bits. The fixed test seed (32 bytes of 0x42), synthetic record, path, challenge, and context reproduce the exact public values in testdata/hardened-v1-vectors.json. TestDeterministicHardenedV1Vectors passed. This verifies deterministic fixture consistency; it is not a production challenge-generation test.

Domain ID and README claims

Hardcoded ProtocolDomain = 202610041

The constant is constrained in the circuit, so it does provide a fixed protocol-domain value for this circuit version. However, 202610041 is an opaque numeric literal with no stable semantic identifier visible from the value itself. It should be mapped to a documented, immutable protocol identifier before interoperability or deployment. Do not change it in-place: changing a circuit constant changes the relation and requires a new circuit/verifying-key version and regenerated keys. This validation did not change it.

README claim review

Most README claims match the code: exact challenge/context binding, recipient/deployment/version binding, policy digest checks, fixed-size diagnosis membership, Merkle-path authentication, signature relation, and explicit range checks are implemented. README correctly states that freshness, external registry trust, and UsedNullifiers storage are not implemented.

One claim needs a stated registry assumption: README says the nullifier is “stable for that record across requests.” The circuit computes it from PatientSecret and the Merkle path-derived index. It does not guarantee a record keeps the same index across root updates, nor prevent duplicate insertion. Stability therefore holds only if the authoritative registry guarantees unique records and stable indices. Without that registry invariant, the same record could yield a different nullifier after reinsertion/reindexing. This is an observation, not a source edit.

Decision

PASS WITH OBSERVATIONS. Compilation, module verification, all tests, and both benchmarks passed. The circuit enforces the tested request/context bindings. External authority/state anchoring and replay state remain unimplemented by design. Before deployment, replace the opaque domain literal with a documented immutable identifier in a separately versioned circuit, and make record-index stability/uniqueness an explicit registry invariant. Challenge freshness and UsedNullifiers remain external.

This report stops at Z7 validation. No Z8, Gateway, Fabric, IPFS, or Federated Learning work was performed.